🎯 Liquidity Hunter Bot — PRO v2.0

> الفلسفة: لا تتبع السعر — اصطد السيولة.
> السوق دائماً يبحث عن الجانب المحاصر. هذا البوت يحلل أين تتجمع الأوامر المعلقة وأين ستُصفَّى المراكز المحاشورة، ثم يتداول معها لا ضدها.

📌 نظرة عامة

في وضع (Perpetuals) الآجلة Binance USDT-M هو بوت تداول كمي يعمل على عقود Liquidity Hunter Bot يستخدم نظام 10 طبقات لتحليل السوق وتوليد إشارات تداول. (محاكاة كاملة بدون أموال حقيقية) Paper Trading مدروسة بناءً على:

- خرائط السيولة (أين تتجمع وقود التصفية)
- تحليل المراكز (من المحاصور Long أم Short؟)
- معدل التمويل (من يدفع ثمن الاحتفاظ بالمركز؟)
- Open Interest (يتراكم أم ينفجر؟)
- نسبة Long/Short (تباعد الأذكياء عن الجمهور)

> ⚠ تحذير: هذا النظام للتعلم والاختبار الورقي فقط. لا تستخدمه بأموال حقيقية إلا بعد التحقق من +100 صفقة موثقة بتوقع إيجابي.

🏗 المعمارية — 10 طبقات

```
السوق (Binance Futures API)
        |
        ▼
┌──────────────────────────────────────────┐
|  Layer 1 — Scanner                       |
|  فلترة كل USDT-M Perpetuals بحجم + OI + extremity |
└──────────────────────────────────────────┘
                    | shortlist (top-N symbols)
                    ▼
┌──────────────────────────────────────────┐
|  Layer 2 — Data Collector                |
|  سحب snapshot كامل: OHLCV, OI, LS Ratio,  |
|  Funding, Liquidations, Taker Buy/Sell   |
└──────────────────────────────────────────┘
                    |
        ┌──────────┴──────────┐
        ▼                     ▼
┌──────────────┐      ┌──────────────────┐
|  Layer 3     |      |  Layer 4         |
|  Liquidity   |      |  Positioning Analyzer |
|  Engine      |      |  6 حالات سوق     |
|  خريطة       |      |  Crowded Long/Short, |
|  التصفيات    |      |  Smart Money Div., |
```

```
│               │     │  OI Exhaustion/Accum.   │
└───────┬───────┘     └─────────┬───────────────┘
        │                       │
        └───────────┬───────────┘
                    ▼

┌──────────────────────────────────────────────┐
│  Layer 5 — Context Layer                     │
│  ذاكرة 24 ساعة — هل تكررت الإشارة؟           │
└──────────────────────────────────────────────┘
                    │
                    ▼

┌──────────────────────────────────────────────┐
│  Layer 6 — Regime Detector                   │
│  Trending / Range / Volatile (ADX-based)     │
└──────────────────────────────────────────────┘
                    │
                    ▼

┌──────────────────────────────────────────────┐
│  Layer 7 — Decision Engine                   │
│  نقاط 0-100 بأوزان خمسة مكونات               │
│  liquidity 30% + positioning 25% + OI 20%    │
│  + funding 15% + price action 10%            │
└──────────────────────────────────────────────┘
                    │ score ≥ 60 → setup
                    ▼

┌──────────────────────────────────────────────┐
│  Layer 8 — Trigger Confirmation              │
│  يتحقق من: volume spike + OI reaction        │
│  + rejection wick (يحتاج 2/3 على الأقل)      │
└──────────────────────────────────────────────┘
                    │ confirmed
                    ▼

┌──────────────────────────────────────────────┐
│  Layer 9 — Trade Generator                   │
│  يولد TradeCard: entry zone, SL, TP1/2/3, RR │
│  يرفض إذا RR < 2.0                           │
└──────────────────────────────────────────────┘
                    │
                    ▼

┌──────────────────────────────────────────────┐
│  Layer 10 — Paper Executor                   │
│  تنفيذ محاكى: slippage + spread + kill-switch │
│  PENDING → TRIGGERED → CLOSED_TP / CLOSED_SL │
└──────────────────────────────────────────────┘

        │                   │
        ▼                   ▼

┌──────────────┐    ┌──────────────────────┐
│  Backtest    │    │  Learning Loop       │
│  Engine      │    │  PerformanceAnalyzer │
│  Walk-forward│    │  AdaptiveWeightsEngine│
└──────────────┘    └──────────────────────┘
```

⚙ آلية عمل دورة الفحص الكاملة

تنفذ الدورة التالية (افتراضي: 300 ثانية) كل N:

```
1. Scanner.scan()
   └ يجلب كل USDT-M perpetuals من Binance
   └ فلتر: حجم 24h ≥ 100M$ + OI ≥ حد أدنى + extremity score مرتفع
   └ يُرجع: shortlist (top N رمز)

2. DataCollector.collect(symbols)
   └ لكل رمز في الـ shortlist: snapshot كامل

3. للكل رمز بالتوازي:
   a. LiquidityEngine.build_map(snap)   → خريطة التصفيات
   b. PositioningAnalyzer.analyze(snap) → حالة السوق
   c. ContextLayer.analyze(snap)        → السياق التاريخي
   d. RegimeDetector.detect(snap)       → نظام السوق

4. DecisionEngine.evaluate(...)
   └ يحسب score 0-100
   └ score < 60   → تجاهل
   └ 60 ≤ score   → WATCH ZONE (تنبيه تيليجرام فقط)
   └ score ≥ threshold + trigger confirmed → SETUP

5. TriggerConfirmation.check()
   └ يتحقق من 3 شروط:
      • Volume Spike (حجم > X× المتوسط)
      • OI Reaction (تغيير OI > حد أدنى)
      • Rejection Wick (ذيل رفض على الكاندل)
   └ يحتاج 2 من 3 على الأقل

6. TradeGenerator.generate()
   └ يحسب: entry_zone, SL, TP1/TP2/TP3
   └ يرفض إذا RR < 2.0

7. PaperExecutor.submit()
   └ يحفظ الصفقة كـ PENDING في DB
   └ يراقب السعر كل tick:
      • السعر دخل الزون → TRIGGERED
      • فات الزون > 3% → CANCELLED
      • مر 8 ساعات → CANCELLED (expired)
      • SL hit → CLOSED_SL
      • TP2 hit → CLOSED_TP

8. Learning Loop (كل 5 صفقات مغلقة)
   └ PerformanceAnalyzer يحلل آخر 30 صفقة
   └ AdaptiveWeightsEngine يعدّل أوزان الـ 5 مكونات
```

⬚ نظام التعلم الذاتي

آلية العمل

تلقائياً Decision Engine يُحلّل البوت أداءه الأخير ويعدّل أوزان، بعد كل 5 صفقات مغلقة (TP أو SL):

```
# تفوز أكثر funding_extreme مثال: إذا كانت الصفقات عالية #
# من 0.15 → 0.18 funding يرفع وزن AdaptiveWeightsEngine
```

الإعدادات الحالية

المعامل	القيمة	المعنى
`enabled`	`true`	التعلم مفعّل
`min_sample_size`	10	يبدأ بعد 10 صفقات مغلقة
`adapt_every_n_trades`	5	يتكيّف كل 5 صفقات
`rolling_window`	30	يدرس آخر 30 صفقة فقط

⬚ نظام التسجيل — قاعدة البيانات

الجداول الرئيسية

الجدول	الوصف
`Trade`	كل صفقة: PENDING → TRIGGERED → CLOSED
`TradeOutcome`	نتائج الصفقات المغلقة (للتعلم)
`ScanSnapshot`	نتائج كل دورة فحص
`PortfolioSnapshot`	لقطات الـ equity بمرور الوقت
`LiquidityMap`	خرائط التصفية المحفوظة

دورة حياة الصفقة

```
PENDING    → السعر لم يدخل الزون بعد
    ├── TRIGGERED    → دخل الزون، نفذ الدخول
    │       ├── CLOSED_TP    → ضرب TP2 ✅
    │       └── CLOSED_SL    → ضرب SL ❌
    └── CANCELLED    → فات الزون (>8h) أو انتهت المدة (<3%)
```

⛔ إدارة المخاطر — قواعد صارمة

القاعدة	القيمة
رأس المال الورقي	$10,000
مخاطرة لكل صفقة	1% من الـ equity
الحد الأقصى للمراكز المفتوحة	3
الحد الأقصى للخسارة اليومية	5%
أقصى خسائر متتالية	3
الحد الأدنى لـ Risk/Reward	2.0 (يُفضَّل 2.5)
slippage الدخول	0.05%
slippage الوقف	0.1%
spread	0.02%

🖥 الواجهة — Dashboard

واجهة ويب مبنية بـ NiceGUI على المنفذ 8090:

الصفحة	المحتوى
Overview	KPIs (Equity, PnL, Win Rate, Open Positions, Kill Switch) + آخر الصفقات المغلقة + آخر الإشارات
Scanner	الفحص الأخير مع جميع المقاييس shortlist
Signals	تحليل Decision Engine لكل رمز مع breakdown المكونات
Trades	الصفقات المفتوحة + كل التاريخ
Backtest	مُشغِّل backtest من الواجهة
Settings	عرض كل الإعدادات من `config.yaml`

مميزات الـ Dashboard

- عداد تنازلي للفحص القادم (يتحول أحمر في آخر 15 ثانية)
- تحديث تلقائي عند اكتمال أي دورة فحص
- Uptime counter يعمل مباشرة
- Dark mode بالكامل
- Live progress bar أثناء الفحص

📱 تنبيهات تيليجرام

متى يُرسَل	نوع التنبيه
عند توليد صفقة جديدة	🎯 Setup Alert
رمز بنقاط عالية لكن لم يُأكَّد بعد	👁 Watch Zone
عند إغلاق صفقة بربح	✅ TP Hit
عند إغلاق صفقة بخسارة	❌ SL Hit
كل يوم أحد تلقائياً	📊 Weekly Report
عند تفعيل حماية الخسارة اليومية	🛑 Kill Switch

🚀 التثبيت والتشغيل

المتطلبات

```
Python 3.11+
SQLite (افتراضي) أو PostgreSQL
حساب تيليجرام (للتنبيهات)
```

التثبيت

```
git clone <repo>
cd liquidity_hunter
python -m venv .venv
source .venv/bin/activate   # Linux/Mac
# أو: .venv\Scripts\activate  # Windows
pip install -r requirements.txt
cp .env.example .env
# عدّل .env بـ Telegram credentials
python -m src.main init
```

التشغيل

```
# Dashboard (موصى به)
python -m ui.app
# ثم افتح http://localhost:8090

# CLI فقط
python -m src.main bot

# فحص واحد
python -m src.main scan

# Backtest
python -m src.main backtest --symbol BTCUSDT --days 60 --timeframe 1h
```

هيكل المشروع

```
liquidity_hunter/
├── config.yaml             ← كل الإعدادات (لا أرقام مُرمَّزة في الكود)
├── .env                    ← Secrets (Telegram, DB URL)
├── requirements.txt
├── data/
│   └── liquidity_hunter.db  ← SQLite database
├── logs/                   ← ملفات السجل (rotating)
├── src/
│   ├── core/
│   │   ├── config.py       ← قراءة config.yaml
│   │   ├── database.py     ← SQLAlchemy models
│   │   └── logger.py       ← إعداد السجل
│   ├── exchange/
│   │   └── binance_client.py← Binance Futures API
│   ├── layers/
│   │   ├── scanner.py      ← Layer 1
│   │   ├── data_collector.py← Layer 2
│   │   ├── liquidity_engine.py ← Layer 3
│   │   ├── positioning.py   ← Layer 4
│   │   ├── context.py      ← Layer 5
│   │   ├── regime_detector.py ← Layer 6
│   │   ├── decision_engine.py ← Layer 7
│   │   ├── trigger_confirm.py ← Layer 8
│   │   ├── trade_generator.py ← Layer 9
│   │   └── paper_executor.py  ← Layer 10
│   ├── backtest/
│   │   └── engine.py       ← Walk-forward backtest
│   ├── learning/
│   │   ├── outcome_logger.py
│   │   ├── performance_analyzer.py
│   │   └── adaptive_weights.py
│   ├── alerts/
│   │   └── telegram_bot.py
│   └── main.py             ← Orchestrator + CLI
└── ui/
    ├── app.py              ← NiceGUI Dashboard
    ├── theme.py            ← Design system
    └── components/
        └── widgets.py      ← Reusable UI components
```

خارطة الطريق

المرحلة	الحالة	المحتوى
v1 (الحالي)	✅ مكتمل	Paper Trading + Dashboard + Telegram + Backtest
v2	📋 مخطط	Multi-timeframe confirmations + OI/Funding series في Backtest
v3	🔮 مستقبلي	Auto-execute عبر Binance API (بعد 100 صفقة ورقية + Sharpe ≥ 1.0)
v4	🔮 مستقبلي	نقل لـ VPS + PostgreSQL + Grafana monitoring

⛔ الإصلاحات والتحسينات المطبقة (v2.0)

⛔ إصلاحات حرجة

الملف	المشكلة	الإصلاح
`paper_executor.py`	`pass` في منطق التفعيل — كل الصفقات تظل PENDING ثم تُلغى	استبداله بـ `await self.trigger()` عند دخول السعر الزون
`paper_executor.py`	`missed_entry_max_pct: 0.003` (0.3%) ضيق جداً	رُفع إلى `0.03` (3%) مناسب للعملات المتذبذبة
`paper_executor.py`	انتهاء صلاحية PENDING بعد 4 ساعات فقط	رُفع إلى 8 ساعات
`app.py`	`TypeError: can't subtract offset-naive and offset-aware datetimes` → 500 Server Error	إضافة `tzinfo is None` check قبل طرح التواريخ
`engine.py`	`--timeframe` غير معتمد في Backtest CLI	إضافة `timeframe` parameter كامل
`database.py`	فقدان بيانات `layer_scores` على الـ Trade row	مباشرة submit يُطبَّق بعد patch

⛔ تحسينات

الملف	التحسين
`app.py`	عداد تنازلي للفحص القادم في الـ topbar (أخضر → أصفر → أحمر)
`app.py`	إصلاح جدول Closed Trades (يستخدم `actual_entry_price` الصحيح)
`telegram_bot.py`	Weekly report تلقائي كل أحد
`telegram_bot.py`	Setup alerts منفصلة عن Watch Zone alerts
`main.py`	submit على التريد بعد patch لـ `layer_scores`
`config.yaml`	إعدادات Learning محسّنة: `min_sample_size: 10`, `rolling_window: 30`

⚖ نقاط القوة والضعف

✅ نقاط القوة

- يحلل بنية السوق الحقيقية — retail (لا RSI، لا MACD) لا يعتمد على مؤشرات
- طبقات مستقلة — كل طبقة قابلة للاختبار والاستبدال بشكل منفصل 10
- نظام تعلم ذاتي يتكيّف مع ظروف السوق المتغيرة كل 5 صفقات
- تلقائي يحمي رأس المال يومياً kill-switch
- Backtest walk-forward — لا curve fitting
- لا أرقام مُرمَّزة في الكود — كل الإعدادات في `config.yaml`

- واجهة dashboard كاملة + Telegram alerts

⚠ نقاط الضعف

- مصدر بيانات وحيد (Binance فقط) — انقطاع API = توقف كامل
- لا يدعم Multi-timeframe بعد — يحلل timeframe واحد فقط
- Backtest لا يشمل OI/Funding Series تاريخية — قد يبالغ في التقدير
- لا يدعم Partial Take Profit — يغلق عند TP2 بالكامل حالياً
- التعلم الذاتي يحتاج صفقات مغلقة — لن يبدأ قبل 10 صفقات فعلية
- SQLite لا يتحمل load عالي عند الوصول المتزامن من UI + Bot

⌧ المتغيرات البيئية (.env)

```
# Database
DATABASE_URL=sqlite+aiosqlite:///data/liquidity_hunter.db
# PostgreSQL for VPS:
# DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/liquidity_hunter

# Telegram
TELEGRAM_BOT_TOKEN=your_bot_token_here
TELEGRAM_CHAT_ID=your_chat_id_here

# Environment
ENV=development
```

⚙ الإعدادات الرئيسية (config.yaml)

```
scanner:
  scan_interval_seconds: 300
  top_n_to_monitor: 15
  min_quote_volume_24h_usd: 100_000_000
  funding_extreme_threshold: 0.0005

decision_engine:
  min_score_to_signal: 60
  min_score_full_size: 75

paper_executor:
  initial_capital_usd: 10_000
  risk_per_trade_pct: 0.01
  max_concurrent_trades: 3
  missed_entry_max_pct: 0.03       # ← 3% مناسب للعملات المتذبذبة
  daily_max_loss_pct: 0.05
  daily_max_consecutive_losses: 3

learning:
  enabled: true
  min_sample_size: 10
```

```
  adapt_every_n_trades: 5
  rolling_window: 30
```

فقط Paper Trading Mode — آخر تحديث: مايو 2026